

SASDS — Client Demo Script

Presenter Guide — Live Demo & Feature Walkthrough

Duration: ~20-25 minutes

Format: Slide intro (5 min) → Live Demo (15 min) → Q&A (5 min)

Pre-Demo Checklist

- [] Backend running: `cd backend && source venv/bin/activate && python -m uvicorn app.main:app --reload --port 8000`
- [] Frontend running: `cd frontend && npm run dev -- --host 0.0.0.0`
- [] Browser open to `http://localhost:5173`
- [] Verify green "Backend is running successfully!" badge in top-right
- [] Clear any previous generated projects
- [] Close unnecessary browser tabs
- [] Increase browser zoom to 125% for audience visibility

Part 1: Opening & Problem Statement (3 min)

Talking Points

"Thank you for joining today. I'm excited to walk you through SASDS — the Single Agent Software Development System — an AI-powered IDE that fundamentally changes how software is built."

"Let me start with a question: How long does it typically take your team to go from a product requirement to a working prototype?"

→ Pause for audience response

"For most teams, the answer is days to weeks. With SASDS, we're bringing that down to minutes."

The Problem We Solve

Traditional Development

Requirements → Specs → Design → Code → Test → Debug
2-4 weeks for an MVP
Needs senior developers
Manual code review cycles

With SASDS

Requirements → **One Click** → Working Code
5-10 minutes for a functional prototype
Anyone can describe what they need
AI-powered auto-review in seconds

Debugging consumes 40-60% of dev time

Self-healing code that fixes its own bugs

"SASDS isn't just another code generator. It's a complete AI-powered development environment — think of it as having a senior developer, a QA engineer, and a code reviewer, all working simultaneously and available 24/7."

Part 2: What Makes SASDS Different (2 min)

Talking Points

"The market has tools like GitHub Copilot, Cursor, and ChatGPT. Here's what makes SASDS fundamentally different:"

Existing Tools vs. SASDS — Key Differentiators

Capability	GitHub Copilot	ChatGPT / GPT-4	Cursor AI	SASDS
Code Generation	Line-by-line autocomplete	Copy-paste from chat	File-level suggestions	Full project generation (multi-file)
Real-Time Streaming			Partial	Live token-by-token streaming to IDE
Built-in IDE	Needs VS Code	No IDE	Fork of VS Code	Full browser-based IDE (zero install)
Self-Correction				Automatic test-detect-fix loop
Auto-Pilot		Manual prompts		Proactive project-wide scanning
Analysis				In-browser terminal via WebSocket
Integrated Terminal	Separate		Separate	
File Management	Separate		Limited	Full file tree with CRUD
Context-Aware Chat	Limited	No project context	Good	Knows your open file + project structure
Deployment	N/A	N/A	N/A	One-command Docker deploy

"The key innovation is the SINGLE AGENT architecture — one intelligent agent handles requirements analysis, code generation, testing, review, and self-correction in one unified pipeline. No context switching, no copy-pasting between tools."

Part 3: Live Demo (15 min)

Demo 1: Project Generation — From Idea to Code (5 min)

Script

"Let me show you the magic. I'm going to describe a project in plain English, and SASDS will build it — right now, in real-time."

Actions

1. **Point to the Requirements panel** (left sidebar)
2. **Type this requirement** (or paste — but typing builds suspense):

Build a REST API **for** a task management application **with**:

- **User** model **with** name, email, **and** creation date
- Task model **with** **title**, description, status (pending/**in**-progress/done), priority (low/medium/high), **and** assigned **user**
- CRUD endpoints **for** both users **and** tasks
- Filter tasks **by** status **and** priority
- Input validation **on all** endpoints

3. **Click "1. Analyze"**

Script (while analysis runs)

"First, SASDS analyzes the requirements using AI. It's not just parsing keywords — it's understanding the intent, identifying data models, suggesting API endpoints, and even flagging what might be missing."

4. **Show the analysis results** — point out:

- Modules identified
- Entities with attributes
- API endpoints auto-detected
- Missing information flagged

"Notice it even identified things we didn't explicitly state — like needing a database and proper error handling. That's the AI thinking like a real architect."

5. **Click "2. Generate"**

Script (while code streams)

"Now watch the File Explorer on the left — files are appearing in REAL-TIME as the AI generates them. This isn't a batch process. You're watching the AI think and write code, token by token."

6. **Point to files appearing** in the explorer as they stream in
7. **Click on main.py** to show code in the editor

"Every file is production-ready — proper imports, type hints, error handling, docstrings. This isn't scaffolding — this is deployable code."

8. Click through 2-3 generated files to show variety

Demo 2: Code Refinement — Surgical AI Edits (2 min)

Script

"Now let's say the client wants a change. Instead of regenerating everything, we can make surgical modifications."

Actions

1. Open **main.py** (or the main API file)
2. Click the **Wand icon** (Refine button)
3. Type:

Add a health **check** endpoint at /health that returns the server status **and** current timestamp

4. Click **Apply**

Script

"The AI understands the existing code context. It doesn't overwrite the file — it surgically adds the new endpoint while preserving everything else. And it explains what it changed."

5. Point to the **explanation** the AI provides
6. Show the **modified code** with the new endpoint added

Demo 3: AI Agent Chat — Your Smart Pair Programmer (2 min)

Script

"What if a junior developer doesn't understand part of the code? Instead of going to Stack Overflow or asking a colleague, they can ask the built-in AI agent."

Actions

1. Switch to **"AGENT CHAT"** tab
2. With a file open in the editor, type:

Explain how the task filtering works **and** suggest how I could **add** pagination

Script

"Notice — the agent KNOWS which file is open. It's not a generic ChatGPT conversation. It has full context of your project structure and the specific file you're looking at. This is context-aware intelligence."

3. **Show the response** — it should reference specific code from the open file
-

Demo 4: Auto-Pilot — Proactive Code Intelligence (3 min)

Script

"This is one of our most innovative features. Traditional tools react to problems. SASDS PREVENTS them."

Actions

1. **Click the "Auto-Pilot" button** in the top header
2. **Wait for analysis to complete**

Script (while scanning)

"Auto-Pilot is now scanning EVERY file in the project — looking for bugs, security vulnerabilities, code quality issues, and improvement opportunities. Think of it as having a senior code reviewer available on demand."

3. **Show the results:**

- **High severity** findings (security issues)
- **Medium severity** findings (potential bugs)
- **Low severity** findings (style improvements)
- **Improvement recommendations**

"Each finding includes the exact file, line number, description, and a specific fix recommendation. This level of automated code intelligence simply doesn't exist in current tools."

Demo 5: Interactive Terminal (1 min)

Script

"SASDS includes a full terminal — right in the browser. No separate window, no SSH setup."

Actions

1. Click on the **Terminal section** at the bottom
2. Run a few commands:

```
ls
echo "Hello from SASDS terminal!"
python --version
```

"This is a real PTY-backed terminal connected via WebSocket. You can install packages, run tests, start servers — anything you'd do in a regular terminal. It's a complete development environment in your browser."

Demo 6: File Management (1 min)

Script

"You have full control over the project structure."

Actions

1. **Right-click on a folder** in the File Explorer
2. **Show the context menu:** New File, New Folder, Rename, Delete
3. **Create a new file** → README.md

"Create, rename, delete — full file management without leaving the IDE."

Part 4: Architecture & Innovation Highlights (3 min)

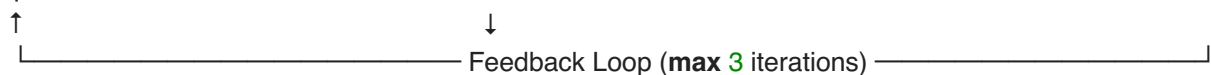
Talking Points

"Let me quickly walk through the technical innovations that power SASDS:"

Innovation 1: Single-Agent Pipeline

"Most AI coding tools use disconnected features. SASDS has a unified 5-stage pipeline where one agent handles everything — requirements analysis, code generation, testing, review, and self-correction. This eliminates context loss between stages."

Requirements → **Code** Generation → Test Creation → **Code** Review → Self-Correction



✂ Innovation 2: Custom Streaming Protocol

"We built a custom streaming protocol that allows the frontend to display code AS IT'S BEING GENERATED — file by file, line by line. This isn't available in any other tool. The user sees exactly what the AI is thinking in real-time."

Innovation 3: Self-Correction Engine

"When tests fail, SASDS doesn't just report the error — it automatically reads the failing file, sends it back to the AI with the error output, writes the fix, and re-runs the tests. Up to 3 iterations. Code that heals itself."

Innovation 4: Proactive Auto-Pilot

"Unlike reactive tools that wait for you to ask, Auto-Pilot proactively scans your entire project for security vulnerabilities, bugs, and improvements — like a 24/7 senior developer reviewing your code."

Innovation 5: Zero-Install Browser IDE

"No VS Code installation. No extension setup. No environment configuration. Open a browser → Start building. That's the onboarding experience."

Part 5: Technology Stack Summary (1 min)

"Under the hood, SASDS is built on enterprise-grade technology:"

Layer	Technology
AI Engine	Google Gemini 2.5 Flash
Backend	FastAPI (Python) — async, high-performance
Frontend	React 18 + TypeScript + Monaco Editor
Deployment	Docker Compose — one command to deploy
CI/CD	GitHub Actions — automated testing & builds
Database	SQLite (dev) / PostgreSQL (prod)

Part 6: Business Impact & ROI (1 min)

 Talking Points

Metric	Impact
Time to Prototype	Reduced from weeks → minutes
Developer Onboarding	New team members productive on day 1
Code Quality	Consistent, AI-reviewed code across all projects

Bug Detection	Proactive scanning catches issues before production
Development Cost	Reduced need for senior developers on routine tasks
Knowledge Gaps	Context-aware chat eliminates Stack Overflow dependency

"The ROI isn't just in faster coding — it's in eliminating entire categories of problems: inconsistent code quality, slow onboarding, missed security vulnerabilities, and the constant context-switching that kills developer productivity."

Part 7: Closing & Q&A (2 min)

Script

"To summarize — SASDS transforms how software is built by combining:"

1. **Natural language to production code** — in minutes
2. **Real-time streaming** — full transparency into AI decisions
3. **Self-healing code** — automatic bug detection and fixing
4. **Proactive security** — Auto-Pilot catches vulnerabilities before they ship
5. **Zero-install, browser-based** — works anywhere, for anyone

"We're not replacing developers. We're giving them superpowers."

"I'd love to take your questions. And if you'd like, I can generate any project you describe — right now, live."

→ Open the floor for Q&A. Offer to do a live generation based on the audience's suggestion.

Backup Demo Ideas (if Q&A is slow)

If you have extra time or the audience wants to see more:

1. **"Name any app idea"** — type it live and generate
2. **Show code review** — submit the generated code for AI review
3. **Show GitHub sync** — push the project to a repo
4. **Show Docker setup** — explain the one-command deployment

Common Questions & Answers

Question	Answer
<i>"Does it work with existing codebases?"</i>	Yes — the chat agent and Auto-Pilot can analyze existing projects. Code refinement modifies existing files.
<i>"What AI model does it use?"</i>	Google Gemini 2.5 Flash — optimized for speed and quality. The model is configurable.

<i>"Is the generated code production-ready?"</i>	It generates well-structured code with error handling and validation, but we recommend human review for production deployments.
<i>"What languages does it support?"</i>	It can generate code in any language — Python, JavaScript, TypeScript, Go, Rust, etc. The AI handles the language selection based on requirements.
<i>"How does it compare to Copilot?"</i>	Copilot autocompletes single lines/functions. SASDS generates entire projects, reviews them, tests them, and fixes them autonomously.
<i>"Is the data secure?"</i>	All processing happens through the configured Gemini API. No code is stored externally. Everything runs in your own infrastructure.
<i>"Can it be deployed on-premise?"</i>	Absolutely — it's fully containerized via Docker. Just docker compose up.

"Thank you! SASDS — Turning ideas into software, one prompt at a time."